

Ранжирование и переранжирование

ШАД, Рекомендательные системы
2025/26, лекция 3

Даня Ткаченко

Руководитель службы ML-сервисов,
Яндекс Лавка

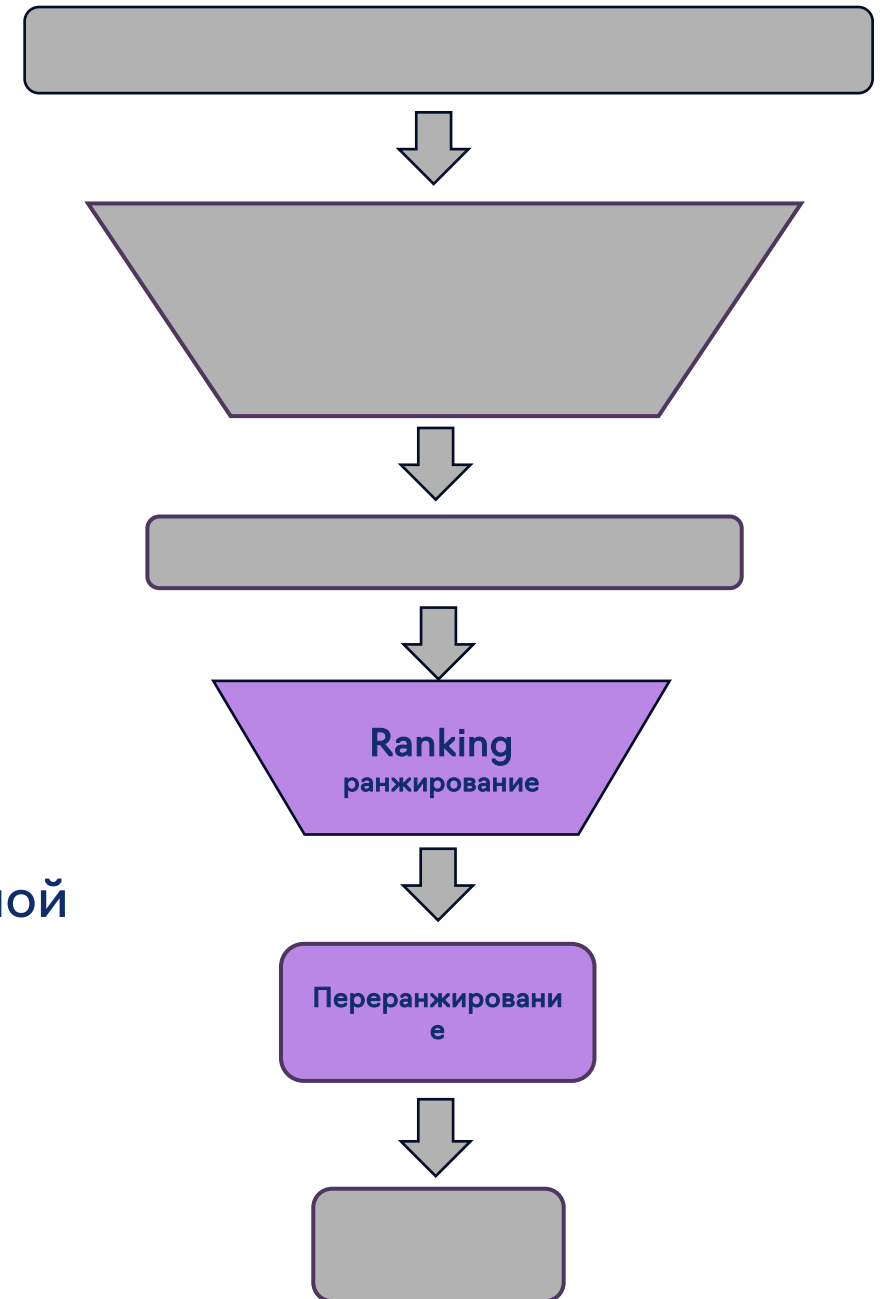
План занятия

На лекции:

1. Истинная релевантность и ее прокси
2. Метрики и схема оценки
3. Лоссы
4. Переранжирование

На семинаре:

1. Посмотрим несколько алгоритмов коллаборативной фильтрации
2. Попробуем обучать ранжирование с разными трюками



Истинная релевантность и ее прокси

Часть 1



Вернемся к прошлой лекции

На прошлом занятии мы рассмотрели классическую постановку задачи и предположили, что существует истинная полезность айтема для фиксированного пользователя:

$$u(i)$$

И если бы ограничений не было, мы бы искали:

$$\operatorname{argmax}_{i \in J} u(i)$$

Но пространство J обычно слишком большое, так что мы строим приближение этого на каком-то выбранном подмножестве -- это кандидатогенерация.

Вернемся к прошлой лекции

На прошлом занятии мы предположили, что существует истинная полезность айтема для фиксированного пользователя:

И если бы огр

На практике (даже если мы можем ее удачно определить), найти данные для обучения невозможно

Но пространство
строим приближение этого на каком-то выделенном подмножестве -- это кандидатогенерация.

Наивный вариант

Обучение по explicit-данным

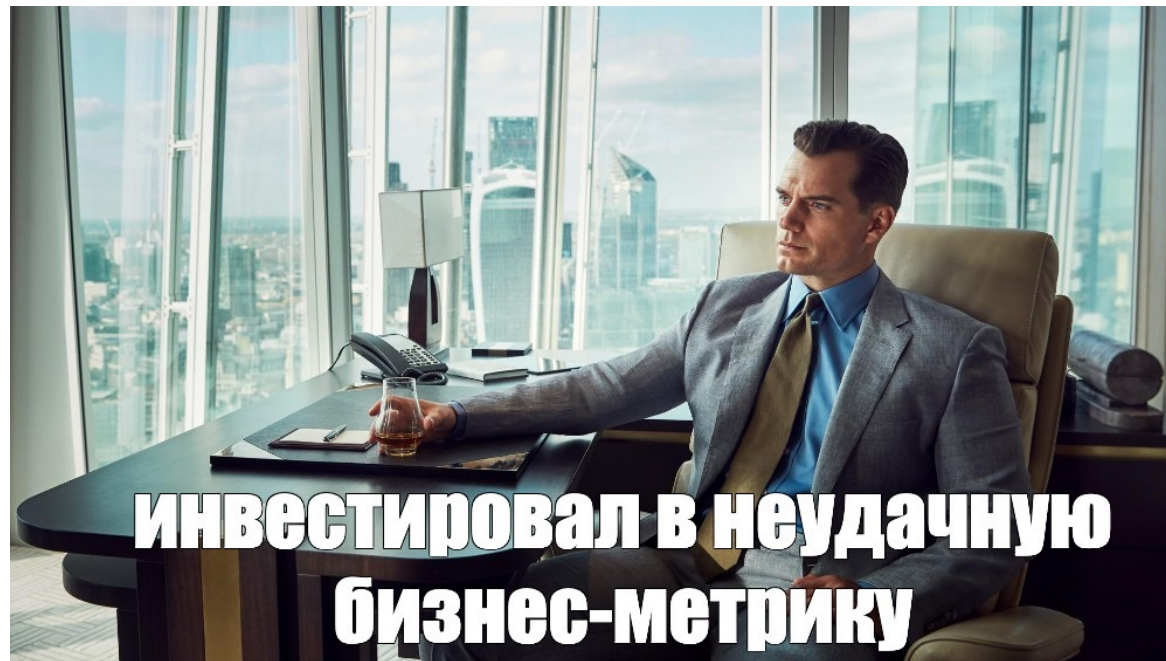
- Посадим людей размечать их собственные рекомендации
- Дорого и плохо
- Данные будут золотые, а нужно нам будет много
- Помимо всех традиционных biases, речь про которые будут дальше, тут появится сильный bias в аудиторию разметчиков
- Другой вариант -- система со встроенным самостоятельным оцениванием
 - риски selection bias
 - Полный разрыв с метриками бизнеса
- В реальности обучение по explicit-данным бывает только в задаче поиска либо в академических статьях

Обучение по implicit-данным

- Будем обучаться на том, как пользователи взаимодействуют с нашей системой
- Нужно как-то спроксировать релевантность:
 - Покупка или добавление в корзину
 - Клик
 - Функция от времени просмотра
- Почему такая модель не будет предсказывать истинную релевантность?

Проблемы обучения по implicit-данным: Неконсистентность прокси

- Мы могли плохо спроксировать цели бизнеса
- Сами цели бизнеса могли быть неудачными



Проблемы обучения по implicit-данным: Логгирующая политика

- Есть данные только про взаимодействие пользователя с айтемами из самой системы
- Более того, только с теми, которые были показаны

Вывод

- Мы можем предсказать только условный $P(\text{click}|\text{show})$, а не $P(\text{click})$
- При этом строго говоря мы обусловлены еще и текущей логгирующей политикой системы

Логгирующая политика =
предыдущая модель + бизнес-
ограничения

Проблемы обучения по implicit-данным: Feedback Loop & Exposure Bias

- Exposure bias -- это и есть наша обусловленность текущей логирующей политикой
- Если с этим ничего не делать, есть угроза Feedback Loop'a:
 - Модель влияет на показы
 - Показы влияют на данные
 - Данные влияют на следующую модель
- Частое следствие:
 - Вырождение системы

Логгирующая политика =
предыдущая модель + бизнес-
ограничения

Проблемы обучения по implicit-данным:

Slate Nature of Data

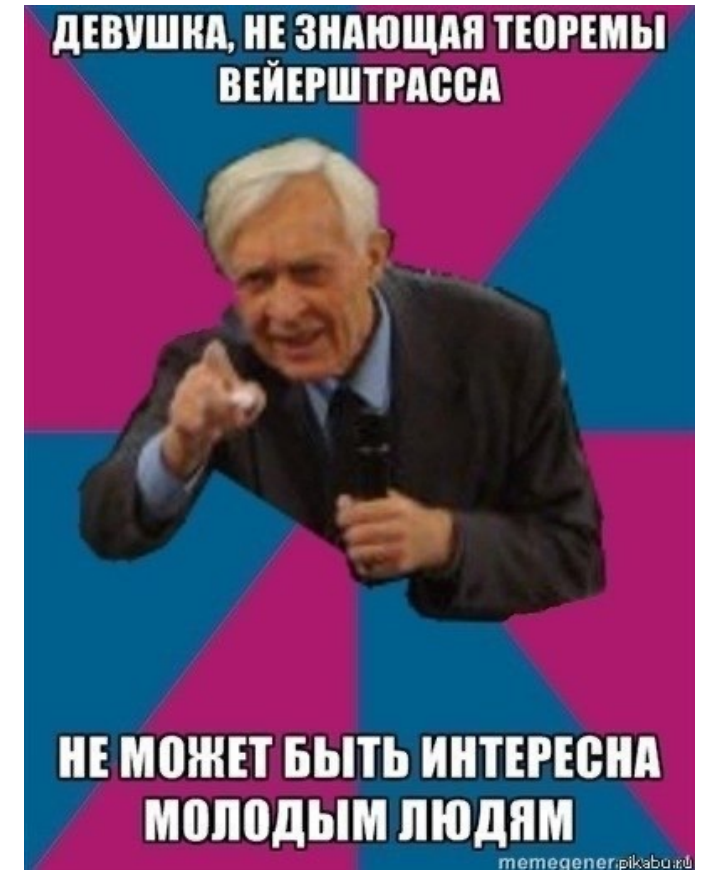
- В реальности пользователь видит целый slate, а не один айтем
 - А иногда даже часть слейта
- Поэтому зачастую еще более строго можно говорить, что мы предсказываем $P(\text{click}|\text{show}, \text{slate})$

Проблемы обучения по implicit-данным: Slate Nature of Data

- В реальности пользователь видит целый slate, а не один айтем
 - А иногда даже часть слейта
- Поэтому зачастую еще более строго можно говорить, что мы предсказываем $P(\text{click}|\text{show}, \text{slate})$

Отсюда очевидное наблюдение, которые многие упускают при обучении ранжирующих моделей

- Если пользователь не видел айтем
- Ака айтем не был в слейте
- Такой айтем не может быть хорошим негативом для ранжирования



Я так же к негативам не из слейта отношусь

Метрики и схема оценки

Часть 2



Вернемся к прошлой лекции

На прошлом занятии мы обсуждали важность Recall'a как метрики кандидатогенерации

Важно понимать, что Recall плохо подходит как метрика ранжирования:

- Мы работаем с уже очень небольшим множеством кандидатов, реколл которого максимально большой с предыдущего шага
- На этапе ранжирования нам больше интересны метрики, которые показывают качество упорядочивания, чем качество покрытия

$$\text{Recall@K} = \frac{1}{|R|} \sum_{k=1}^K rel_k$$

Зафиксируем лейблы

Мы начинаем говорить про метрики, так что тут важно явно проговорить, какие у нас лейблы
Все взаимодействия пользователя можно разбить по slate'ам = выдачам

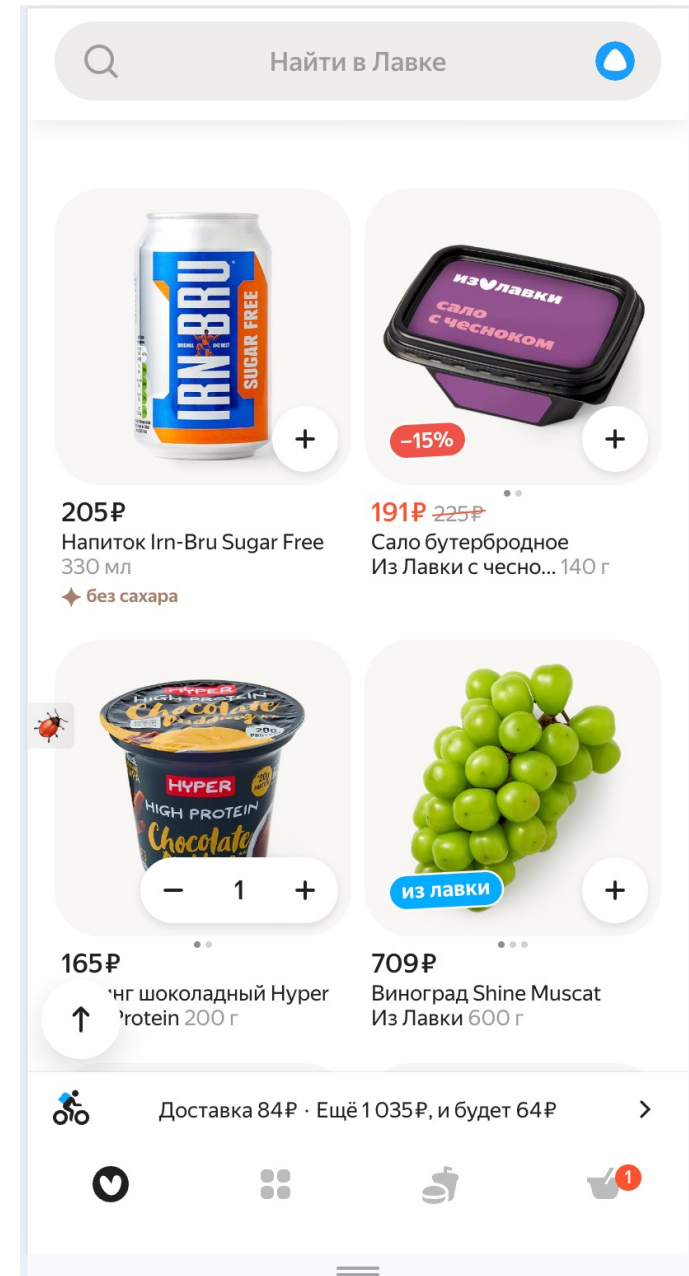
В каждом слейте есть:

- Позитивы -- позитивные взаимодействия из логирующей политики
- Негативы -- ими будем считать те объекты, которые пользователь видел, но с которыми не совершал позитивного взаимодействия по результатам этого показа

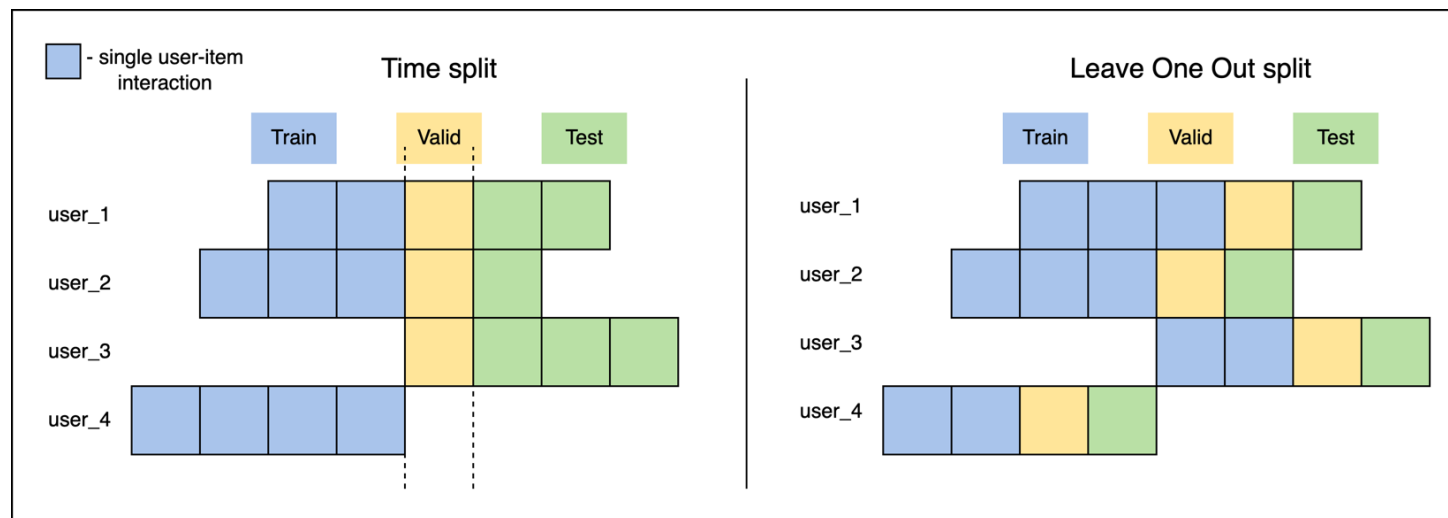


Зафиксируем лейблы

Проговорим еще раз: это не настоящие негативы истинной релевантности, если пользователь не кликнул на айтем, это не значит, что у него небольшая истинная релевантность, это значит, что у него небольшая веростность клика при условии показа



Зафиксируем сплит



<https://habr.com/ru/companies/sberbank/articles/867296/>

При supervised-обучении мы всегда тренируемся на одном наборе данных, а считаем метрики на другом

В задаче рекомендаций важно понимать, что единственный правильный сплит -- временной

Простое объяснение:

- В вашем датасете почти наверняка будут фичи, которые так или иначе будут проксировать популярность айтема
- Если разбивать случайно или использовать какие-то другие стратегии сплита кроме временной, таргет неизбежно протечет через такие фичи

Зафиксируем сплит

Отдельно про применимость leave-one-out

Часто в академических статьях можно увидеть, что авторы использовали для валидации такой вид сплита
Стоит настороженно относиться к результатам этой статьи

88 papers in RecSys conferences (2020 – 2022)

No. papers	Percentage	Train/test split
30	34%	Random split
22	25%	Leave-one-out
17	19.5%	Single time point
15	17%	Simulation-based online
4	4.5%	Sliding window

<https://personal.ntu.edu.sg/axsun/slides/RecSys2024-Review.pdf>

AUC@K

Мы обговорили, что нас не интересует Recall, а что же нас тогда интересует?

Насколько хорошо модель умеет упорядочивать позитивные взаимодействия относительно негативных

По счастливому стечению обстоятельств AUC показывает ровно это

$$\text{AUC} = \frac{1}{|P||N|} \sum_{i^+ \in P} \sum_{i^- \in N} \mathbf{1}[f(u, i^+) > f(u, i^-)]$$

Positional bias

Реальность такова, что пользователь обычно склонен купить что-то из того, что вы покажете на более высокой позиции

Важно понимать: это вносит проблему на уровне данных, а не на уровне обучения модели

Мы, однако, можем постараться немного это подкорректировать при помощи правильного выбора метрик

Посмотрим на наши варианты

MRR@K

Mean Reciprocal Rank

Пусть

$$k^* = \min\{k : rel_k > 0\}$$

—позиция первого релевантного элемента в слейте.

Тогда:

$$RR = \frac{1}{k^*}$$

Если релевантных в топ-K нет — обычно $RR = 0$.

Если есть множество слейтов Q :

$$MRR = \frac{1}{|Q|} \sum_{q \in Q} RR_q$$

MRR@K

Mean Reciprocal Rank

Пусть

$$k^* = \min\{k : rel_k > 0\}$$

—позиция первого релевантного элемента в слейте.
Тогда:

$$RR = \frac{1}{k^*}$$

Если релевантных в топ-K нет — обычно $RR = 0$.
Если есть множество слейтов Q :

$$MRR = \frac{1}{|Q|} \sum_{q \in Q} RR_q$$

Значение = 1, если первый позитив всегда был на первом месте и очень невелико, если он был всегда где-то далеко

Такая метрика подходит именно для случая, когда из природы продукта мы знаем, что позитив у нас будет ровно один

В определенной степени корректирует позиционные эффект для такого сценария

MAP@K

Mean Average Precision

Мы взвешиваем релевантность на позиции по Precision@K

$$AP = \frac{1}{|R|} \sum_{k=1}^K \text{Precision@k} \cdot rel_k$$

Потом усредняем по слайтам

$$MAP = \frac{1}{|Q|} \sum_{q \in Q} AP_q$$

.

MAP@K

Mean Average Precision

Мы взвешиваем релевантность на позиции по Precision@K

$$AP = \frac{1}{|R|} \sum_{k=1}^K \text{Precision@k} \cdot rel_k$$

Потом усредняем по слайтам

$$MAP = \frac{1}{|Q|} \sum_{q \in Q} AP_q$$

Некоторый учет позиции происходит из-за того, что в нормальной системе Precision@K более-менее убывает с ростом позиции

Метрика работает по факту только для моделей классификации

Исторически используется для ранжирования, хотя по сути довольно сильно штрафует, если наша модель не предсказала какой-то айтем как позитив

NDCG@K

Normalized Discounted Cumulative Gain

Это самая важная метрика ранжирования и по большей части современный стандарт

$$CG@K = \sum_{k=1}^K g(rel_k) \quad DCG@K = \sum_{k=1}^K \frac{g(rel_k)}{\log_2(k+1)}$$

$$IDCG@K = \sum_{k=1}^K \frac{g(rel_k^{ideal})}{\log_2(k+1)} \quad NDCG@K = \frac{DCG@K}{IDCG@K}$$

Что важно:

- Метрика может быть все еще довольно высокой, если мы нашли только пару позитивов, но зато все сверху
- Позволяет (и требует) свое определение Gain'a (например, для есom можно впихивать цену позитива)

Практический момент

Все эти метрики подразумевают какое-то усреднение, и обычно под этим усреднением имеется в виду просто усреднение по слайтам

На практике бывает полезно дополнительно усреднять по пользователям

Просто объяснение:

- Если у вас есть какой-то дисбаланс между двумя когортами пользователей (например, новички и старички сервиса)
- Среди запросов большинство будут от старичков
- Можно не увидеть плохую картинку на новичках таким образом

Давайте посмотрим на специфические лоссы ранжирования, которые задизайнены растить именно такие метрики

Лоссы

Часть 3



Виды лоссов для задачи LTR

Learning-to-Rank

Обычно рассматривают три группы лоссов

Pointwise

Pairwise

Listwise

В зависимости на каком множестве айтемов этот лосс считается

Давайте посмотрим на каждую из них

Pointwise

Базовая идея:

- Игнорируем все проблемы, про которые говорили выше
- И просто предсказываем поточечный таргет aka
- Моделируем нашу релевантность напрямую

Самый типичный пример таргета:

- 1 если товар был добавлен в корзину
- 0 если товар был просмотрен, но не добавлен

На выходе получаем какое-то подобие $P(\text{add}|\text{show})$.
Особенно в свете того, что чаще всего используют BCE

$$\mathcal{L}_{\text{BCE}} = - \sum_{i \in S} \left[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \right]$$

Pointwise

Почему pointwise недостаточно

- Не учитывает структуру слейта и порядок в частности
- Не согласован напрямую с AUC / NDCG

Pairwise

Базовая идея:

- Учимся тому, что позитив должен быть выше негатива
- Строго говоря, моделируем статистику, индуцирующую порядок, соответствующий наблюдаемому порядку
- Лосс всегда определяется как какая-то сумма по парам позитив-негатив (обычно из одного слейта, но стреляйте себе в ноги, если хочется)
- Интуитивно согласовано с AUC, так какк доля правильно отранжированных пар

Посмотрим на примере BPR

Bayesian Personalized Ranking

$$\mathcal{L}_{\text{BPR}} = - \sum_{i^+ \in P} \sum_{i^- \in N} \log \sigma(f(u, i^+) - f(u, i^-))$$

Выводится по ММП при моделировании вероятности того, что позитив больше негатива через сигмоиду

Самый стандартный попарный лосс, большинство попарных лоссов являются его надстройками:

- Разного рода перевзвешивания
- Более сложный семплинг пар

Концептуально соответствует RankNet для поисковых систем

LambdaRank

Идея: подобрать лосс так, чтобы градиенты были пропорциональны изменению метрики (например, ΔNDCG).

$$\mathcal{L}_{\text{LambdaRank}} = \sum_{(i,j)} \Delta\text{Metric}_{ij} \cdot \log(1 + \exp(-s_{ij}(f_i - f_j)))$$

Не оптимизируем NDCG напрямую, но взвешиваем pairwise-градиенты вкладом в метрику.

При помощи пристального взгляда видно, что это BPR с весами

Listwise

Базовая идея:

- Учимся сразу правильно располагать весь слайт
- Моделируем структуру слайта по сути

Модель учится сразу располагать правильно весь набор айтемов и таким образом учитывает порядок явно

Softmax-over-slate

Самый простой лосс, часто используется в силу удобства в имплементации и стабильности

$$P(i \mid S) = \frac{\exp(f(u, i))}{\sum_{j \in S} \exp(f(u, j))}$$

$$\mathcal{L}_{\text{Slate}} = - \sum_{i \in S} y_i \log P(i \mid S)$$

LambdaRank как listwise-лосс

Можно заметить, что LambdaRank по сути является листовым лоссом, поскольку в весах оказываются функции от всего слейта (дельты NDCG или другой метрики, которая рассчитывается на всем слейте)

Про Yetirank отдельно говорить не будем, он по сути является некоторого рода надстройкой над LambdaRank, которая умеет майнить более сложные пары

Резюмируя

Pointwise соответствует
моделированию самой релевантности

Pairwise -- моделированию ее
наблюдаемого порядка

Listwise -- моделированию структуры
слейта

Резюмируя

Pointwise соответствует
моделированию самой релевантности

Все они оптимизируют прокси при
фиксированной политике показа.
Как нам научиться влиять на политику так, чтобы
нивелировать bias'ы?

Listwise -- моделированию структуры
слейта

Переранжирование

Часть 4



Соберем в кучку

По ходу лекции мы уже успели ввести несколько разных bias'ов, например:

- Selection bias
 - Это когда пользователи склонны оценивать только определенные айтемы
 - Избегаем при помощи implicit-обучения
- Positional bias
 - Про то, что пользователи имеют склонность с большей вероятностью покупать то, что было показано на более высокой позиции
 - Частично смягчаем его при помощи top-heavy метрик

Соберем в кучку

- Exposure bias
 - Про то, что в логах появляется только то, что до этого было показано
 - И связанный с ним
- Popularity bias
 - В данных популярных айтемов всегда больше, и выучить что-то про тяжелый хвост модели просто сложнее

Соберем в кучку

- Exposure bias
 - Про то, что в логах появляется только то, что до этого было показано
 - И связанный с ним
- Popularity bias
 - В данных популярных айтемов всегда больше, и выучить что-то про тяжелый хвост модели просто сложнее

Есть еще много разных bias'ов, корректный debiasing рексистем -- отдельная тема. Эти два bias'a, однако, -- основные риски feedback loop'a.

Соберем в кучку

- Exposure bias
 - Про то, что в логах появляется только то, что до этого было показано
 - И связанный с ним
- Popularity bias
 - В данных популярных айтемов всегда больше, и выучить что-то про тяжелый хвост модели просто сложнее

Есть еще много разных bias'ов, корректный debiasing рексистем -- отдельная тема. Эти два bias'a, однако, -- основные риски feedback loop'a.

Мы научились эксплуатировать знания от старой логирующей политики. Давайте посмотрим, как эксплорировать эти знания.

Эксплорейшен

Рандомная квота в выдаче

- В бандитской терминологии соответствует эpsilon-гриди
- На доле эpsilon выдачи будем показывать случайные рекомендации
- В остальных местах будем показывать выдачу нашей модели

Плюсы:

- Точно сильно разнообразивает пул
- Можно усложнять, делая выдачу не рандомной, а выученной на отдельный эксплоративный таргет

Минусы:

- Делает это довольно топорно, нужно аккуратно подбирать сетап, чтобы не сильно навредить бизнес-метрикам

Эксплорейшен

Софтмакс сэмплинг

- Посчитаем софтмакс от наших скоров с некоторой температурой и посэмплим из получившегося распределения без возвращения

Плюсы:

- Довольно бережно с точки зрения бизнес-метрик
- Легко подобрать хороший сетап
- Есть хороший трюк для быстрого расчета

Минусы:

- Переранжирует только существующий топ
- Либо нужно внедрять на всех этапах

Практический момент

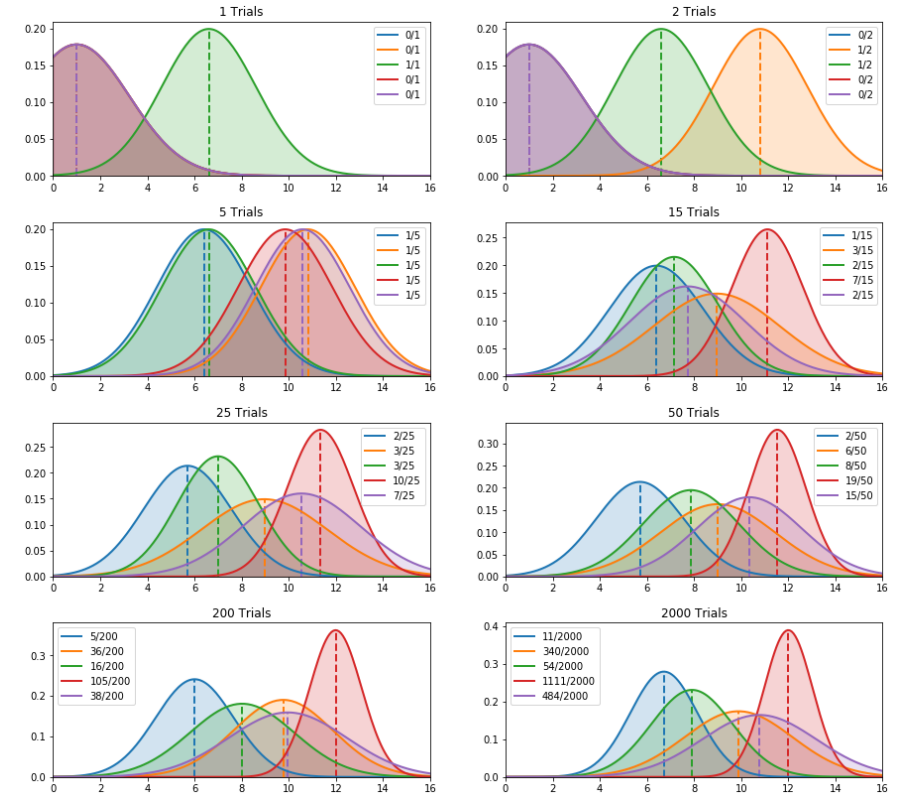
С большой долей вероятности двух этих трюков вам хватит для того, чтобы пройти первые 95% профита

Дальше интереснее

Напоминание

Базовая идея основных алгоритмов бандитов

- Thompson Sampling: будем поддерживать распределение для вероятности награды на каждой ручке, сэмплить из каждой и брать максимум / сортировать
- UCB: вместо сэмплинга будем добавлять что-то в духе среднеквадратичного отклонения и брать максимум / сортировать уже по нему



Случай рексистем

Бандиты в тех теоремах, которые чаще всего доказывают в курсах по RL, не контекстные – награда не зависит от того, кто дергает за ручку, и какая сейчас погода за окном

В случае рекомендательных систем это очень ограничивающее предположение, поэтому используют бандитов, которые умеют это учитывать, -- контекстных

На это и многое другое мы посмотрим в отдельной лекции по RL дальше в курсе, тк это выходит за рамки классической постановки задачи

Небольшой практический хак

На практике, однако, мы можем заиспользовать идеи бандитов с некоторыми допущениями, если мы просто представим их как переранжирующий слой после работы основного алгоритма

У нас уже есть наша функция, которая предсказывает какой-то скор, пусть она будет наградой. Но нам нужно моделировать все распределение, по одной точке это сделать сложно.

Идея: наш предикт – это уже среднее этого распределения. Нужно откуда-то взять дисперсию для сэмплинга.

Небольшой практический хак

На практике, однако, мы можем заиспользовать идеи

На практике можно за дешево:

- Просто поддержать N моделей (ансамбль) и считать дисперсию отсюда по выборке
- В нейросетевых случаях можно включить дропаут и сделать несколько forward-pass'ов
- Обучить честный линейный UCS поверх выхода сложной модели

Первые два варианта стоят примерно ничего по разработке

распределения. Нужно откуда-то взять дисперсию для сэмплинга.

Разнообразие

Легко перепутать с эксплорейшном. Тут меняем логирующую политику не ради самого изменения логирующей политики, а чаще всего по бизнес-соображениям

Легко предположить, что если у нас есть похожие товары, у одного пользователя скорее всего будут похожие скоры для этого товара. Если у нас таких товаров очень много, мы скорее всего не захотим, чтобы они занимали всю выдачу.

Подобрали для вас



165₽

Пудинг ванильный Hyper High Protein 200 г



165₽

Пудинг Hyper High Protein карамельный 200 г

Методы лечения Бизнес-ранкеры

Самый частый и дешевый вариант – жадно переранжируем так, чтобы удовлетворить какому-то бизнес-правилу:

- Хороший пример – не больше двух товаров из одной категории подряд
- Или на каждом префиксе должно быть распределение категорий как можно более близкое к равномерному

Методы лечения

MMR и похожие методы

MMR – Maximum Marginal Relevance

Строим слейт жадно, балансируя релевантность и разнообразие.

На шаге k выбираем айтем:

$$i_k = \arg \max_{i \notin S} \left[\lambda \cdot \text{relevance}(u, i) - (1 - \lambda) \cdot \max_{j \in S} \text{similarity}(i, j) \right]$$

где:

- S – уже выбранные элементы слейта
- relevance – скор модели
- similarity – мера похожести айтемов (обычно похожесть по контентному эмбеду)
- Лямбда – гиперпараметр

Методы лечения DPP

DPP – Determinant Point Process

Будем строить выдачу из условно 100 элементов кусками по 10 штук
Для этого зададим распределение на 10-элементных подмножествах,
в котором вероятности будут пропорциональные определителю
соответствующий подматрицы такой матрицы.

$$K_{ij} = \text{relevance}(i) \text{relevance}(j) \text{similarity}(i, j)$$

Теперь просто посэмплим из этого распределения.

Почему так:

- Только интуитивно – определитель соответствует квадрату объема параллелепипеда, натянутого на вектора, из которых составлена подматрица
- Если какие-то два коллинеарны или близки к этому, то объем уменьшается
- Если мы увеличиваем длину ростом релевантности, он растёт

На практике

Такое распределение, конечно, очень тяжело построить явно. Но существуют алгоритмы, которые могут делать приближенные сэмплинги из этого распределения за разумное время.

Обсудить сейчас их мы, к сожалению, не успеем.

В реальности DPP редко применяется в продакшене, так как даже приближенные алгоритмы обычно предполагают явный подсчет и спектральное разложение матрицы близостей, что довольно затратно по ресурсам. И все чаще всего обходятся простыми методами

Неклассические постановки

Если мы решаем задачу ранжирования в секвенциональной постановке (о ней будет дальше в курсе) в виде предсказания следующего позитивного айтема, то мы можем строить слейт авторегрессионно, что снимает проблемы с разнообразием

Существуют и диффузионные подходы к генерации целого слейта, но они по большей части остаются экзотикой

Что мы узнали сегодня

- Много всего про ранжирование
- Что обучать ранжирование нужно обязательно с негативами по показам, умоляю
- Про то, как бороться с разными bias'ами внутри логирующей политики
- В частности про то, что пул должен быть разнообразным
- И про то, как немного менять логирующую политику в нужную сторону



Спасибо!

Даня Ткаченко,
Руководитель службы ML-сервисов Яндекс Лавки

Белград 2026

